

Universidade Federal de Alagoas (UFAL)

Disciplina: Programação 2

Projeto: Rede de Relacionamentos Jackut - Milestone 1

Autor: Yuri Raphael Mota De Araújo Barbosa

1. INTRODUÇÃO

Este relatório descreve as decisões arquiteturais e os padrões de projeto adotados no desenvolvimento do Milestone 1 do sistema Jackut. O objetivo principal do design foi garantir a alta coesão, o baixo acoplamento e o encapsulamento estrito das entidades, cumprindo 100% dos testes de aceitação propostos pelo cliente através da ferramenta EasyAccept.

2. ARQUITETURA GERAL E PADRÕES DE PROJETO

O sistema foi arquitetado utilizando uma separação clara entre a camada de interface com os testes (Facade) e a camada de lógica de negócios (Controllers e Model). Para evitar o antipadrão *God Class* (uma única classe centralizando todas as regras do sistema), as responsabilidades foram divididas utilizando os seguintes padrões de projeto:

2.1. Padrão Facade (Fachada)

A classe Facade foi implementada como o único ponto de entrada para os testes do EasyAccept. Ela atua como um escudo para o modelo de domínio, escondendo a complexidade interna do sistema.

- **Justificativa:** O padrão Facade permite que a interface exigida pelos testes (os 11 comandos específicos) seja atendida sem poluir as regras de negócio. A Facade não contém lógica própria; ela apenas recebe os parâmetros brutos (Strings) e delega as operações para os controladores especializados.

2.2. Padrão Singleton

Os controladores lógicos do sistema (UsuarioController, SessaoController, AmizadeController, RecadoController) foram implementados utilizando o padrão Singleton.

- **Justificativa:** Como o sistema atua como uma aplicação centralizada (sem injeção de dependência complexa como o Spring Boot neste momento), o Singleton garante que exista apenas uma instância global de gerenciamento para cada domínio. Isso impede inconsistências no armazenamento em memória durante a execução das *User Stories* e facilita o acesso aos dados entre os diferentes módulos sem a necessidade de instanciar classes repetidamente.

2.3. Padrão Controller (Separação de Responsabilidades / SRP)

Em vez de um controlador monolítico (SistemaController), a lógica foi fragmentada aplicando o Princípio da Responsabilidade Única (SRP) do SOLID:

- **PersistenciaController:** Isola completamente as operações de I/O (Input/Output). É a única classe que conhece a existência do arquivo jackut.dat.
- **SessaoController:** Focada exclusivamente em autenticação e segurança (geração de UUIDs).
- **AmizadeController e RecadoController:** Mediam as regras de negócio que envolvem a interação entre dois usuários diferentes.
- **UsuarioController:** Gerencia o ciclo de vida (CRUD) e o armazenamento base dos perfis.

3. DECISÕES DE MODELAGEM DE DOMÍNIO (DOMAIN-DRIVEN)

O pacote model foi desenhado para ser o mais autossuficiente e encapsulado possível.

- **Entidade Recado:** Em vez de trafegar os recados como tipos primitivos puros (String), foi criada uma entidade Recado. Isso prepara o sistema para futuras expansões (como adicionar data de envio ou status de leitura) sem quebrar as assinaturas dos métodos existentes.
- **Encapsulamento de Coleções:** As coleções internas da entidade Usuario (como a lista de amigos e a fila de recados) nunca são expostas diretamente. O método getAmigos(), por exemplo, retorna uma Collections.unmodifiableList, garantindo que nenhuma classe externa possa corromper o estado das amizades (adicionando ou removendo itens burlando as regras de negócio).

- **Uso de LinkedHashMap/LinkedHashSet:** Para garantir o determinismo necessário para os testes de aceitação (que esperam retornos em ordens específicas), optou-se por utilizar implementações baseadas em *Linked* nas coleções principais do sistema, preservando a ordem de inserção dos dados cronologicamente.

4. DIAGRAMA DE CLASSES

Abaixo encontra-se a representação UML da arquitetura do sistema, demonstrando a interação entre a camada de testes (Facade) e as camadas de controle e modelo, garantindo o encapsulamento das regras de negócio.

